CSI101 COMPUTER SYSTEMS

Lab 1 — Comparing Numbers

Assembly Programming · Practical application 11.1 — Compare two numbers

2-hour lab session · Visual Studio 2022 · Win32 Console Application · x86 platform

This first lab is about talking to the C library from inside an assembly block, and about the one mechanism the processor gives you for making a decision: compare, then jump. You will run the lecture program, then extend it twice.

What you will be able to do

- Call `printf` and `scanf_s` from inside an `_asm` block, pushing arguments right to left and cleaning up four bytes per argument.
- Use `CMP` to set the flags and `JE` / `JG` to act on them.
- Bring several branches back together at a single exit point, so the printing happens in one place.
- Extend a two-way comparison to three values using the running-maximum pattern.
- Drive an assembly block repeatedly from a C++ loop, and pass a result back through a shared variable.

How the session is organised

0:00 – 0:25 Part A. Type the lecture program in, build it, run it, and read the notes.

0:25 – 1:10 Task 1. Extend the program.

1:10 – 1:50 Task 2. Extend it again.

1:50 – 2:00 Work through the checklist at the end and ask about anything that did not build.

> **TYPE IT, DO NOT PASTE IT**
>
> Typing catches the syntax details that reading skips over — the missing comma, the wrong register width, the label with no instruction after it. The solutions at the end of this handout are for checking your work, not for copying.

Before you start

1. Open Visual Studio 2022 and choose **File > New > Project > Console App (C++)**. Give it the name shown at the top of this handout.
2. Set the **Solution Platforms** dropdown on the toolbar to **x86**. It defaults to x64, where the `_asm` keyword is rejected outright with error C4235.
3. Replace the generated `main` with the Part A listing, and make sure `#include <cstdio>` is at the top — `printf` and `scanf_s` come from there.
4. Press **Ctrl + F5** to build and run without the debugger, so the console window stays open when the program finishes.

> **IF THE PLATFORM IS WRONG**
>
> With the platform left at x64 the compiler reports that the `_asm` keyword is not supported on this architecture (C4235). Inline assembly exists on 32-bit x86 and nowhere else.

Part A — the lecture program

The program prompts for two integers, reads them with `scanf_s`, compares them, and prints one of three messages. Every string and every variable is declared in C++, because inline assembly cannot define data of its own — there is no `DB`, `DW` or `DD` available to you. The input, the comparison and the output all happen inside the `_asm` block.

Declarations (C++ owns all the data)

```
#include <cstdio>

// ------------------------------------------------------------------------
//  All data is declared in C++.  Inline assembly cannot define data of its
//  own - no DB, DW or DD - so every string and every variable lives here and
//  the _asm blocks simply refer to them by name.
// ------------------------------------------------------------------------
char msgFirst[]     = "Enter the first number: ";
char msgSecond[]    = "Enter the second number: ";
char msgThird[]     = "Enter the third number: ";
char msgEqual[]     = "\nThe numbers are equal.\n";
char msgFirstBig[]  = "\nThe first is larger.\n";
char msgSecondBig[] = "\nThe second is larger.\n";
char format[]       = "%d";

int first, second, third;
```

The program

```
void partA()
{
    printf("--- Part A: compare two numbers ---\n\n");

    _asm {
        // ---- prompt for and read the first number ----------------------
        lea     eax, msgFirst       // eax = address of the prompt string
        push    eax                 // one argument for printf
        call    printf
        add     esp, 4              // 1 argument x 4 bytes = 4

        lea     eax, first          // scanf_s must WRITE into first,
        push    eax                 //   so it needs the address, not the value
        lea     eax, format         // "%d"
        push    eax                 // format string is pushed last
        call    scanf_s
        add     esp, 8              // 2 arguments x 4 bytes = 8

        // ---- prompt for and read the second number ---------------------
        lea     eax, msgSecond
        push    eax
        call    printf
        add     esp, 4

        lea     eax, second
        push    eax
        lea     eax, format
        push    eax
        call    scanf_s
        add     esp, 8

        // ---- compare, then converge on one exit ------------------------
        mov     eax, first
        cmp     eax, second         // subtracts second from first, keeps flags
        je      equal               // ZF = 1  -> the two are equal
        jg      firstBigger         // SIGNED greater-than; JA would be wrong here

        lea     eax, msgSecondBig   // fell through: second must be larger
        jmp     show

    firstBigger:
        lea     eax, msgFirstBig
        jmp     show
```

```
    equal:
        lea     eax, msgEqual

    show:                           // one printf, reached by all three paths
        push    eax
        call    printf
        add     esp, 4
    }
}
```

Expected output

```
Enter the first number: 17
Enter the second number: 9

The first is larger.
```

What to notice

- `LEA` loads the **address** of something; `MOV` would load its contents. `printf` needs the address of the first character of the string, not the character itself.
- `scanf_s` has to write into your variable, so it needs the variable's address too — which is why `LEA` appears there as well.
- `printf` and `scanf_s` use the `__cdecl` convention: arguments are pushed **right to left**, and the **caller** removes them afterwards. One argument means `add esp, 4`; two arguments mean `add esp, 8`.
- `JG` is the **signed** greater-than jump. With `JA` any negative number would appear to be the larger one, because `JA` reads the carry flag rather than the sign and overflow flags.
- Three paths, one `printf`. Each branch loads a different message address into `EAX` and jumps to `show`. Less code, and one place for a mistake to hide instead of three.

Task 1 — the largest of three numbers

What to write

- Add a third prompt and read a third integer.
- Report which of the three is the largest.
- Do all the comparing in assembly. C++ may print the answer with an ordinary `printf`.

Expected output

```
Enter the first number: 10
Enter the second number: 3
Enter the third number: 25

The largest of the three is 25.
```

Hints

- Use the **running maximum** pattern: keep the largest value seen so far in `EAX`, and compare each new candidate against it. Two comparisons are enough for three values.
- The shape repeats: `cmp eax, second` / `jge skipIt` / `mov eax, second` / `skipIt:` — then the same again for the third value.
- A label must always be followed by an instruction. Put the `mov largest, eax` after your final label rather than leaving the label dangling at the end of the block.
- Test it with negative numbers — `-5`, `-20`, `-1` should give `-1`. If it gives `-20`, you have used an unsigned jump.

Task 2 — keep asking until the two numbers match

What to write

- Repeat the two prompts until the user types the same number twice in a row.
- Count the attempts using an `INC` instruction inside the assembly block.
- When the numbers finally match, report how many attempts it took.

Expected output

```
Enter the first number: 4
Enter the second number: 9

Not equal yet - try again.

Enter the first number: 7
Enter the second number: 7

Matched after 2 attempt(s).
```

Hints

- Let C++ own the repetition — a `while (equalFlag == 0)` loop around one `_asm` block is much easier to read than a loop built entirely from jumps, and it is the division of labour the lecture recommends.
- `INC` works directly on a memory operand, so `inc attempts` needs no register at all.
- To set a C++ flag from assembly without leaving a label dangling: `xor edx, edx` first, `jne storeFlag` when the values differ, `mov edx, 1` when they match, then `storeFlag: mov equalFlag, edx`.

- The `_asm` block appears once in the compiled program even though it runs many times, so its labels are defined once. You do not get a duplicate-label error.

Before you leave

- The Solution Platform reads **x86**, not x64.
- Every `printf` and `scanf_s` call is followed by an `ADD ESP` of four bytes per argument.
- Signed jumps (`JG`, `JGE`, `JL`, `JLE`) are used throughout, and you have tested with negative input.
- Every label is followed by an instruction.
- You have tested all three outcomes: equal, first larger, second larger.

If something will not build

- `_asm not supported on this architecture` — the platform is still x64. Change it to x86.
- `improper operand type` — the operand size is ambiguous. Add `DWORD PTR`, or check you have not mixed `eax` with `al`.
- `undeclared identifier` inside a block — a misspelt variable name, or one not in scope. Names are **case sensitive** inside `_asm`, even though instruction mnemonics are not.
- The program builds but crashes *after* the block — the stack is unbalanced, or `EBX`, `ESI`, `EDI` or `EBP` was not restored.
- Right answer, wrong sign — an unsigned jump was used on signed data, or `SHR` where `SAR` was needed.